TRAVEL & STAYS

A beginner-friendly guide to what your app does and how its code fits together

In one sentence
Travel & Stays is a two-sided travel marketplace: travellers find and book stays and local tours, while hosts and guides manage what they offer.

This guide is for understanding the big picture, not memorising every line. Read it once, then use it as speaking notes for your presentation.

Your app uses: React for the screens, React Router for page navigation, Supabase for accounts and data, and Paystack for payments (configured in test mode).

1. What the app does

The project is a marketplace for travel in Nigeria. Different kinds of users have different jobs in it:

- **Travellers:** search for places to stay and tours, view details, save stays, make bookings, pay, and review completed stays.
- **Hosts:** create and manage property listings, set availability, and handle booking requests.
- **Tour guides:** create tours and manage tour bookings and departures.
- **Admins:** monitor activity and manage trust-and-safety reports or verification.
- **Trust and safety:** verification, reports, safety information, and trip-sharing features help users travel more safely.

A useful user journey

A traveller opens the site, searches stays, opens a listing, signs in if needed, books dates, and receives feedback. Behind the screen, the React page displays the interface, Supabase handles account/data tasks, and a toast can confirm an action.

The main technologies

Technology	Plain meaning
React	Builds the interface from reusable components.
Vite	Runs the local development server and bundles the app.
React Router DOM	Connects URL paths to the page components shown.
Supabase	Provides database, authentication, file storage, and live features.
Paystack	Payment service; this project describes payments as test mode.

Important: the browser-side route check controls what the interface displays, but database security also matters. Supabase Row Level Security (RLS) is used to restrict access to data.

2. How the code is organised

Folders group files by their job. This makes it easier to find code and change one feature without putting the entire app in one huge file.

Place	What it is for
index.html	The browser?s starting HTML page. It contains the empty root element where React is mounted.
src/main.jsx	The entry point: starts React and wraps the app in shared providers and the router.
src/App.jsx	The route map: says which page appears for each URL and which routes need protection.
src/pages/	Full screens, such as Home, Login, Trips, Profile, and Admin.
src/components/	Reusable UI parts, such as Navbar, Footer, ListingCard, and ProtectedRoute.
src/context/	Shared app-wide information and actions, such as auth, toast messages, saved items, and presence.
src/hooks/	Reusable React logic, such as loading listings or handling a cooldown.
src/lib/	Data-access functions and helpers, including the Supabase client, bookings, dates, and formatting.
supabase/	Database schema, ordered migrations, demo data, and backend functions.

The start-up path

index.html ? main.jsx ? App.jsx ? matching page ? its components and helpers

Why are providers wrapped around the app?

A provider is like a shared service desk. If AuthProvider wraps the app, pages beneath it can ask for the signed-in user. ToastProvider lets pages ask to show a short notification. Wrapping makes shared data available without threading it manually through every component as props.

Why do some components sit inside other components?

A parent component can contain child components. The parent chooses when and where a child appears, and can pass it information. For example, a layout can contain a navbar, the current page, and a footer. The nesting describes the UI structure and who can access shared context.

3. Auth, services, routes, and toast

Authentication (auth)

Auth answers ?Who is signed in?? In this app, AuthProvider restores and listens for the Supabase session, loads the user?s profile, and shares a convenient user object with the rest of the app. A login page sends credentials to the auth service; when sign-in succeeds, the app can show pages meant for that user.

A protected route checks access before rendering a page. For example, a profile page needs a signed-in user; host pages also check for the host role. If access is missing, the app redirects or shows an explanation.

Service / lib code

A ?service? is a common name for code that performs an operation, often data access. This project puts many of these functions in src/lib rather than using a folder literally named services. For example, lib/supabase.js creates the Supabase client; other lib files group work for bookings, listings, reviews, and payments. Pages can call these helpers instead of repeating the same database work.

Toast notifications

A toast is a small temporary message. ToastProvider keeps a list of active messages, displays them, and removes them after a timer or when the user dismisses one. A page can call the shared show function after an action, such as saving a stay. This gives feedback without interrupting the user with a dialog.

One simple flow

User clicks Save ? page calls saved-item logic ? data is updated ? page asks the toast context to show ?Saved? ? ToastProvider renders the notification.

Async work

A database or network request takes time, so it is asynchronous. JavaScript continues doing other work while waiting. The code can use a Promise with .then(...) or an async function with await. The UI often tracks a loading state so it can show a spinner while the request is in progress.

Example of the app?s structure (simplified)

```
<BrowserRouter>
  <ToastProvider>
    <AuthProvider>
      <App />
    </AuthProvider>
  </ToastProvider>
</BrowserRouter>
```

4. JavaScript and React words

Function: A reusable set of instructions. It can take inputs, do work, and return a result.

Parameter: A named input in a function definition. In function greet(name), name is a parameter.

Argument: The actual value supplied when calling a function, such as greet("Sam").

Callback: A function passed to another function so it can be called later. A timer callback runs when its time is up.

Asynchronous / async: Work that may finish later, such as a database request. It does not freeze all JavaScript while waiting.

Promise: An object representing a result that may be available later. .then(...) handles its result; await waits inside an async function.

Array: An ordered list of values, such as a list of toast messages.

.map(): Transforms each array item into a new item. React commonly maps a list of data to a list of UI components.

Component: A function that returns a piece of user interface, usually written with JSX.

JSX: HTML-like syntax inside JavaScript that React uses to describe the interface.

Props: Inputs passed from a parent component to a child component.

State: Data React remembers. When state changes, React updates the displayed interface.

Hook: A React function such as useState or useEffect that lets a component use React features.

Context: A way to share values with many nested components without passing props at every level.

Render: React?s process of creating or updating what the user sees.

DOM: The browser?s object representation of the web page. React DOM connects React to that browser page.

Route: A rule connecting a URL, such as /login, to a component such as the Login page.

API: A defined way for one piece of software to ask another for data or an action.

Small map example

If toasts is an array, toasts.map(toast => <Toast key={toast.id} toast={toast} />) means: ?For every toast in the array, make one Toast component.? The key helps React keep track of each item when the list changes.

5. Commands and presentation notes

Run these from the client folder

Command	What it does
`npm install`	Downloads the packages listed for this project. Usually needed after first setup.
`npm run dev`	Starts the local development server. Keep the terminal open and use the local URL it prints.
`npm run lint`	Checks the project for common code and style problems.
`npm run build`	Creates an optimized production build, usually in dist/. It does not start the development server.
`npm run preview`	Previews the production build locally after building.

For running locally, the usual command is npm run dev. The project also needs valid environment settings for its Supabase connection; check client/.env.example and ask your project owner before sharing any secret values.

A short presentation script

?My project is called Travel & Stays. It is a travel marketplace where travellers can find places to stay and local tours, while hosts and guides manage their listings. The app is built with React. React Router connects URLs to pages. Auth manages the signed-in user, Supabase stores and retrieves account and app data, and toast notifications give users quick feedback. The code is organised into pages, reusable components, shared context, hooks, and helper functions so each part has a clear job.?

A quick study plan

- Explain the app in one sentence.
- Point to index.html, main.jsx, then App.jsx and describe the start-up path.
- Choose one feature (auth or toast) and explain its purpose and basic flow.
- Name two JavaScript terms you now understand, such as async and callback.
- Finish by saying npm run dev starts the app locally.

Use the actual code as your source of truth. This guide summarises the project structure and features in the repository README and client code; it is an overview, not a line-by-line code reference.